



CPE 486/586: Machine Learning for Engineers

05 Optimization and Gradient Descent

Fall 2026

Rahul Bhadani

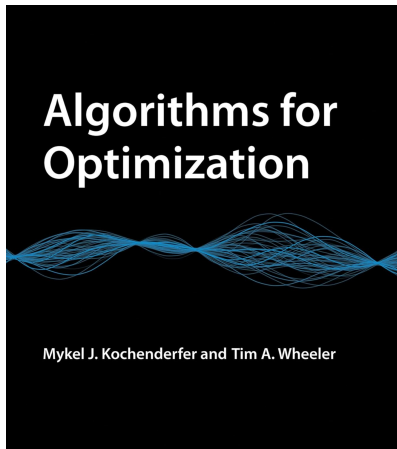


Outline

1. Optimization
2. Constraint Optimization: Lagrange Multiplier
3. Penalty Method in Constraint Optimization
4. Gradient Descent for Optimization
5. Constraint Optimization with Inequality



Optimization



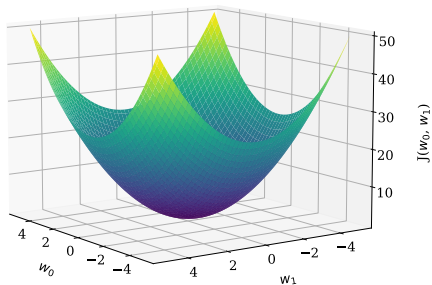
Algorithms for Optimization, second edition,, Chapter 2, 4, 5, 10 (Freely Available)

<https://algorithmsbook.com/optimization/files/optimization.pdf>

What is Optimization?

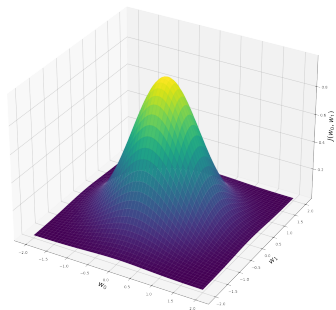
- 1 Maximizing or minimizing some function relative to some set, often representing a range of choices available in a certain situation.
- 2 The function allows comparison of the different choices for determining which might be “best”.

A Function with a Minimum



This function has a single global minimum.

A Function with a Maximum



This function has a single global maximum.

How do we find global maximum/minimum

Finding Maxima/Minima of One Variable

A Function of One Variable: $f(x)$

To find the local maxima and minima of a function f on an interval $[a, b]$:

- ⚡ Solve for $f'(x) = 0$ to obtain critical point(s) c .
- ⚡ Drop from the list any critical points that aren't in the interval $[a, b]$.
- ⚡ $x = c$, is a point of local maxima if $f'(c) = 0$, and $f''(c) < 0$. The point at $x = c$ is the local maxima and $f(c)$ is called the **local maximum** value of $f(x)$.
- ⚡ $x = c$ is a point of local minima if $f'(c) = 0$, and $f''(c) > 0$. The point at $x = c$ is the local minima and $f(c)$ is called the **local minimum** value of $f(x)$.

Example 1

Find the local maxima and local minima of the function

$$f(x) = 2x^3 + 3x^2 - 12x + 5 \quad (1)$$

Example 1

Solution:

$$\frac{\partial f(x)}{\partial x} = 2 \cdot 3x^2 + 3 \cdot 2x - 12 = 0$$

$$6x^2 + 6x - 12 = 0$$

$$x^2 + x - 2 = 0$$

$$x^2 + 2x - x - 2 = 0$$

$$x(x + 2) - 1(x + 2) = 0$$

$$(x + 2)(x - 1) = 0$$

$$\Rightarrow x = -2, x = 1$$

(2)

$$\frac{\partial^2 f(x)}{\partial x^2} = 12x + 6$$

$$\left. \frac{\partial^2 f(x)}{\partial x^2} \right|_{x=-2} = -18 < 0 \quad (3)$$

$$\left. \frac{\partial^2 f(x)}{\partial x^2} \right|_{x=1} = 18 > 0$$

Hence, $x = -2$ for local maximum, and $x = 1$ is for local minimum. $f(-2)$ is local maximum and $f(1)$ is local minimum.

Finding Maxima/Minima of Two Variables

Consider a function $f(x, y)$ of two variables x and y .

If f has a local extremum (or critical point) at (a, b) , and if the first order partial derivatives of f exist there, then

$$\frac{\partial f}{\partial x}(a, b) = 0 \quad \text{and} \quad \frac{\partial f}{\partial y}(a, b) = 0$$

or (4)

$$\nabla f(a, b) = \begin{bmatrix} \frac{\partial f}{\partial x}(a, b) & \frac{\partial f}{\partial y}(a, b) \end{bmatrix} = \begin{bmatrix} 0 & 0 \end{bmatrix} = \mathbf{0}$$

At a critical point, a function could have a local maximum, or a local minimum, or neither.

Hessian Function

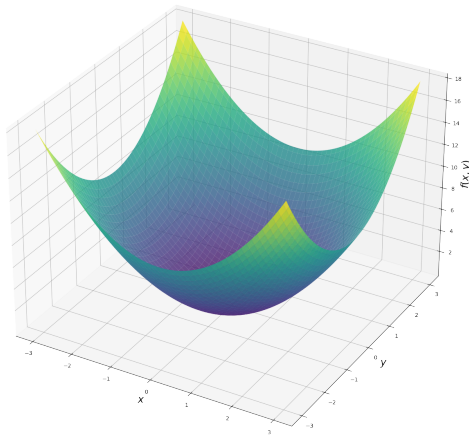
If a $f(x, y, z)$ has critical points (a, b, c) , then in order to check if it is a maximum, minimum or a saddle point, we need to compute Hessian matrix first.

$$H = \begin{bmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} & \frac{\partial^2 f}{\partial x \partial z} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} & \frac{\partial^2 f}{\partial y \partial z} \\ \frac{\partial^2 f}{\partial z \partial x} & \frac{\partial^2 f}{\partial z \partial y} & \frac{\partial^2 f}{\partial z^2} \end{bmatrix}$$

- 1 If all eigenvalues of Hessian function are positive, then H is positive definite, and critical point is local minimum.
- 2 If all eigenvalues of Hessian function are negative, then H is negative definite, and critical point is local maximum.
- 3 If eigenvalues of Hessian function are both positive and negative, then critical point is a saddle point.

Example 2

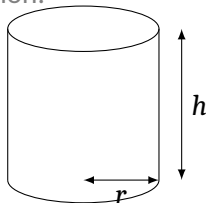
Let $z = f(x, y) = x^2 + y^2$. The only critical point is $(0, 0)$. At $(0, 0)$, f has a minimum.



Example 3

A cylindrical hot water tank is to have a capacity of 4 m^3 . Find the radius and height that would have the least surface area.

Solution:



$$\pi r^2 h = 4 \quad (5)$$

$$h = \frac{4}{\pi r^2}$$

$$\text{Surface Area} = 2\pi r^2 + 2\pi rh$$

Example 3 Continues ...

Solution:

$$S = 2\pi r^2 + 8r^{-1}$$

$$\frac{\partial S}{\partial r} = 4\pi r - 8r^{-2}$$

$$4\pi r - 8r^{-2} = 0$$

$$r^3 = \frac{8}{4\pi}$$

$$\Rightarrow r = 0.860$$

$$\begin{aligned} \frac{\partial^2 S}{\partial r^2} &= 4\pi + 16r^{-3} \\ &= 4\pi + \frac{16}{(0.860)^3} > 0 \end{aligned} \tag{7}$$

Hence $r = 0.860$ gives minimum surface area,
and $S = 2\pi(0.860)^2 + \frac{8}{0.860} = 13.949$

Example 4

Compute local minima/maxima of $f(x, y, z) = x^2 + 2y^2 + 3z^2 - 4x - 6y - 8z + 10$

Solution:

$$\nabla f(x, y, z) = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z} \right) = (2x - 4, 4y - 6, 6z - 8) \quad (8)$$

Setting them to 0, gives $x = 2, y = \frac{3}{2}, z = \frac{4}{3}$.

Computing the Hessian matrix:

$$H = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 4 & 0 \\ 0 & 0 & 6 \end{bmatrix} \quad (9)$$

The Hessian matrix's eigen values are 2, 4, 6 and hence the Hessian matrix is positive definite. Hence, the critical point given by $x = 2, y = \frac{3}{2}, z = \frac{4}{3}$ is arg minimum with minimum value at $f(2, \frac{3}{2}, \frac{4}{3})$.

Constraint Optimization: Lagrange Multiplier

Lagrange Multiplier

The method of Lagrange Multipliers is an excellent technique for finding the global maximum and global minimum values of a function $f(x, y)$ when the values of x and y that need to be considered are subject to some form of constraint, usually expressed as an equation $g(x, y) = 0$.

Problem Formulation

Minimize/Maximize $f(x, y)$

subject to

$$g(x, y) = 0.$$

In such a case, the optimality occurs when $\nabla f + \lambda \nabla g = 0$.

Example 5

Maximize/Minimize

Cost function: $f(x, y) = 81x^2 + y^2$, subject to the constraint $4x^2 + y^2 = 9$

Solution:

Lagrange Multiplier is $L(x, y, \lambda) = f(x, y) - \lambda g(x, y)$ where $g(x, y) = 4x^2 + y^2 - 9 = 0$.

$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} & \frac{\partial f}{\partial y} \end{bmatrix} = [162x \quad 2y], \quad \nabla g = \begin{bmatrix} \frac{\partial g}{\partial x} & \frac{\partial g}{\partial y} \end{bmatrix} = [8x \quad 2y] \quad (10)$$

Example 5 ...

$$\begin{aligned}\frac{\partial f}{\partial x} - \lambda \frac{\partial g}{\partial x} &= 0 \Rightarrow 162x - \lambda 8x = 0 \\ \frac{\partial f}{\partial y} - \lambda \frac{\partial g}{\partial y} &= 0 \Rightarrow 2y - \lambda 2y = 0\end{aligned}\tag{11}$$

From the first equation, $8x(21 - \lambda) = 0 \Rightarrow \lambda = 21$ or $x = 0$. From the second equation, $2y(1 - \lambda) = 0 \Rightarrow y = 0$ or $\lambda = 1$.

If $x = 0 \Rightarrow 4 \cdot 0^2 + y^2 = 9 \Rightarrow y = \pm 3$. Hence the first critical point is $(0, \pm 3)$.

If $y = 0 \Rightarrow 4x^2 + 0^2 = 9 \Rightarrow x = \pm 3/2$. Hence the second critical point is $(\pm 3/2, 0)$.

$f(0, \pm 3) = 9$ and $f(\pm 3/2, 0) = 729/4 = 182.25$.

Example 6

Maximize/Minimize

Cost function: $f(x, y) = 8x^2 + 2y$, subject to the constraint $x^2 + y^2 = 1$

Example 7

Maximize/Minimize

Cost function: $f(x, y, z) = y^2 - 10z$, subject to the constraint $x^2 + y^2 + z^2 = 36$

Penalty Method in Constraint Optimization

Penalty Function

In order to use PyTorch and Gradient Descent Method, we need to reformulate constraint optimization problem using penalty function.

$$\begin{array}{ll}\text{minimize } f(\mathbf{x}) \\ \text{subject to } h(\mathbf{x})\end{array}$$

changes to

$$\begin{array}{ll}\text{minimize } f(\mathbf{x}) + \mu h^2(\mathbf{x}) \\ \text{where } \mu \text{ is a penalty term.}\end{array}$$

An optimal solution theoretically would have $h^2(\mathbf{x})$ close to zero. The square term ensures that the penalty is always non-negative.

Limitations of Penalty Method

- ⚡ If the penalty term is too small, then the constraint may not be satisfied
- ⚡ If the penalty term is too large, then the solution may be ill-condition, potentially leading to NaN (Not a Number).

Why does Penalty Method Work

The penalty term $\mu h^2(\mathbf{x})$ acts as a “soft constraint”, pushing the solution toward the feasible region where $h(\mathbf{x}) = 0$.

Reference:

- 1 https://ocw.mit.edu/courses/15-084j-nonlinear-programming-spring-2004/13b4c46d590cf3df94475e0ae1a815c3_lec10_penalty_mt.pdf
- 2 <https://solmaz.eng.uci.edu/Teaching/MAE206/Lecture14.pdf>
- 3 <https://www.rose-hulman.edu/~bryan/lottamath/penalty.pdf>

Gradient Descent for Optimization

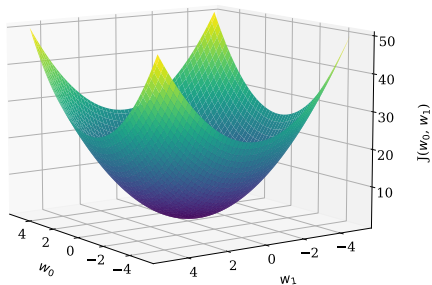
Algorithmic Approach to Minimizing the Cost Function: Gradient Descent

The cost function for linear regression with one variable:

$$J(w_0, w_1) = \frac{1}{2n} \sum_{i=1}^n (y_i - w_0 - w_1 x_i)^2 \quad (12)$$

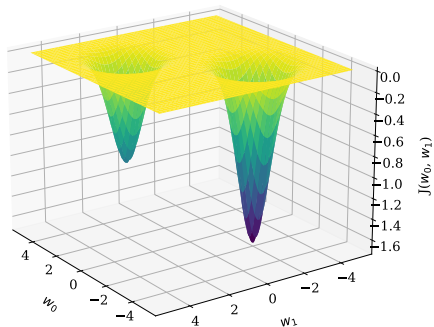
which is quadratic in w_0, w_1 .

Gradient Descent



This function has a single global minimum.

Gradient Descent



This function has two local minima (out of which one is the global minimum).

Gradient Descent Algorithm

We find the minima iteratively:

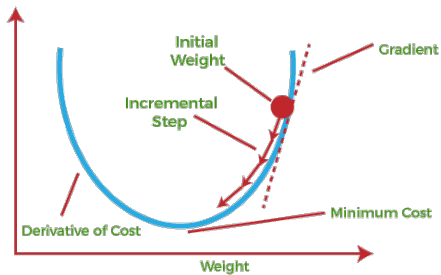
$$\begin{aligned}w_0 &\leftarrow w_0 - \eta \frac{\partial}{\partial w_0} J(w_0, w_1) \\w_1 &\leftarrow w_1 - \eta \frac{\partial}{\partial w_1} J(w_0, w_1)\end{aligned}\tag{13}$$

Gradient Descent Algorithm: Vectorize Form

The partial derivative is the **gradient** (slope towards the minimum). We iteratively calculate the above equation until w_0 and w_1 converge. η is called the learning rate. Succinctly, we can write it as

$$\mathbf{w}(t+1) \leftarrow \mathbf{w}(t) - \eta \nabla J(\mathbf{w}(t)) \quad \text{where} \quad \nabla J(\mathbf{w}) = \begin{bmatrix} \frac{\partial}{\partial w_0} J(w_0, w_1) \\ \frac{\partial}{\partial w_1} J(w_0, w_1) \end{bmatrix} \quad (14)$$

Gradient Descent Visualization



Gradient Descent Algorithm: Example 8

Simple Cost Function

$$J(w_1) = (w_1 - 5)^2 + 10$$

Gradient Descent Algorithm: Example 8

Simple Cost Function

$$J(w_1) = (w_1 - 5)^2 + 10$$

$$\frac{\partial J(w_1)}{\partial w_1} = 2w_1 - 10 \quad (15)$$

Gradient Descent Algorithm: Example 8

Simple Cost Function

$$J(w_1) = (w_1 - 5)^2 + 10$$

$$\frac{\partial J(w_1)}{\partial w_1} = 2w_1 - 10 \quad (15)$$

We could solve it analytically, but currently, we are interested in solving it numerically using a gradient descent algorithm: $w_1 = w_1 - \eta \frac{\partial J(w_1)}{\partial w_1}$.

Gradient Descent Algorithm: Example 8

Simple Cost Function

$$\begin{aligned} J(w_1) &= (w_1 - 5)^2 + 10 \\ \frac{\partial J(w_1)}{\partial w_1} &= 2w_1 - 10 \end{aligned} \tag{15}$$

We could solve it analytically, but currently, we are interested in solving it numerically using a gradient descent algorithm: $w_1 = w_1 - \eta \frac{\partial J(w_1)}{\partial w_1}$.

We start with a few values of w_1 and η .

Batch Gradient Descent

- ⚡ All training data are taken into account to take a single step.
- ⚡ Take the mean of the gradients of all the training examples to update our parameters.
- ⚡ Requires the entire dataset to be available in the memory.

For example, we calculate the gradient of the following:

$$J(w_0, w_1) = \frac{1}{2n} \sum_{i=1}^n (y_i - w_0 - w_1 x_i)^2 \quad (16)$$

Mini-batch Gradient Descent

- ⚡ Instead of the entire dataset: we take small random subsets of the training data (mini-batches) at each iteration.

We sample a random subset $\mathcal{C}_t \subset \{1, 2, \dots, n\}$, $|\mathcal{C}| = b \ll m$, the batch size, and our update rule is

$$\mathbf{w}(t+1) \leftarrow \mathbf{w}(t) - \eta \frac{1}{b} \sum_{i \in \mathcal{C}_t} \nabla J_i(\mathbf{w}(t)) \quad (17)$$

and full gradient is approximated as

$$\mathbb{E} \left[\frac{1}{b} \sum_{i \in \mathcal{C}_t} \nabla J_i(\mathbf{w}) \right] = \nabla J(\mathbf{w}) \quad (18)$$

Stochastic Gradient Descent

- ⚡ Just use a single training example (or data sample) until a sufficient value is reached for $\mathbf{w}$.
- ① Take a training data sample,
 - ② Feed to the equation model,
 - ③ Calculate the gradient,
 - ④ Update the weights,
 - ⑤ Repeated 1-4 for all data samples.

While the batch gradient descent always converges, it is slow and costly. Stochastic gradient descent is good for the large dataset, makes progress faster, but oscillates in terms converging.

PyTorch for Gradient Descent

```
import torch
import matplotlib.pyplot as plt

# Define the cost function J(w_1)
def J(w_1):
    return (w_1 - 5)**2 + 10

# Initialize w_1 with requires_grad=True
w_1 = torch.tensor(0.0, requires_grad=True)
# Set up the optimizer
optimizer = torch.optim.SGD([w_1], lr=0.1)
# Store the values of w_1 for plotting
w_1_values = []
```

```
# Perform gradient descent
for i in range(100):
    # Zero the gradients
    optimizer.zero_grad()
    # Compute the loss
    loss = J(w_1)
    # Backward pass (compute gradients)
    loss.backward()
    # Update parameters
    optimizer.step()
    # Store the current value
    w_1_values.append(w_1.item())
plt.plot(w_1_values)
plt.xlabel('Iteration'), plt.ylabel('$w_1$'), p
```

Automatic Differentiation

Automatic Differentiation

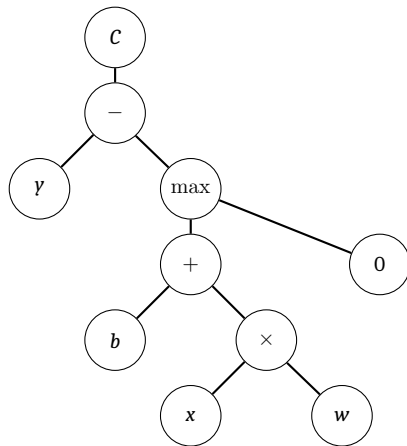
Automatically calculate the derivative of a function by repeatedly applying the chain rule. It can calculate the partial differentiation wrt many inputs. Read more:

- 1 https://www.cs.toronto.edu/~rgrosse/courses/csc321_2018/slides/lec10.pdf
- 2 https://en.wikipedia.org/wiki/Automatic_differentiation
- 3 <https://www.tensorflow.org/guide/autodiff>

Automatic Differentiation Example

We will construct expression graph of $\mathcal{C}(y, wx + b) = y - \max(0, wx + b)$ to understand how automatic differentiation works. Automatic differentiation only works with numerical values, hence, let $y = 5$, $w = 2$, $x = 1$, $b = 1$.

Our target is to calculate $\frac{\partial \mathcal{C}}{\partial w}$.



Automatic Differentiation Example

Let:

$$w_1 = w = 2 \quad w_2 = x = 1$$

$$w_3 = b = 1 \quad w_4 = \gamma = 5$$

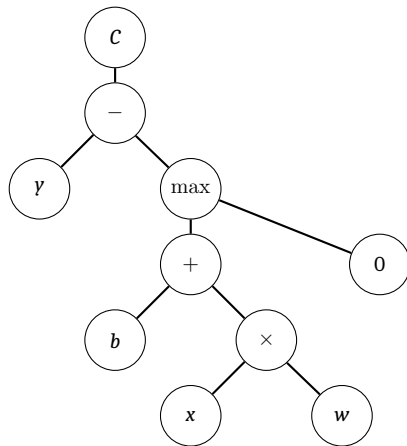
$$w_5 = w_1 w_2 = 2 \quad w_6 = w_3 + w_5 = 3$$

$$w_7 = \max(0, w_6) = 3$$

$$w_8 = w_4 - w_7 = 2$$

$$C = w_8 = 2$$

(19)



Automatic Differentiation Example

$$\frac{\partial w_5}{\partial w_1} = \frac{\partial(w_1 \cdot w_2)}{\partial w_1} = w_2$$

$$\frac{\partial w_5}{\partial w_2} = \frac{\partial(w_1 \cdot w_2)}{\partial w_2} = w_1$$

$$\frac{\partial w_6}{\partial w_5} = \frac{\partial(w_5 + w_3)}{\partial w_5} = 1$$

$$\frac{\partial w_7}{\partial w_6} = \frac{\partial \max(0, w_6)}{\partial w_6} = \begin{cases} 0, w_6 < 0 \\ 1, w_6 > 0 \end{cases}$$

(20)

$$\frac{\partial w_8}{\partial w_7} = \frac{\partial(w_4 - w_7)}{\partial w_7} = -1$$

$$\frac{\partial w_8}{\partial w_4} = \frac{\partial(w_4 - w_7)}{\partial w_4} = 1 \quad (21)$$

$$\frac{\partial \mathcal{C}}{\partial w_8} = 1$$

Apply chain rule:

$$\frac{\partial \mathcal{C}}{\partial w_1} = \frac{\partial \mathcal{C}}{\partial w_8} \times \frac{\partial w_8}{\partial w_7} \times \frac{\partial w_7}{\partial w_6} \times \frac{\partial w_6}{\partial w_5} \times \frac{\partial w_5}{\partial w_1} = ? \quad (22)$$

Terminology (1 of 2): the expression graph

Expression graph. A directed acyclic graph that records how a value is computed from primitive operations.

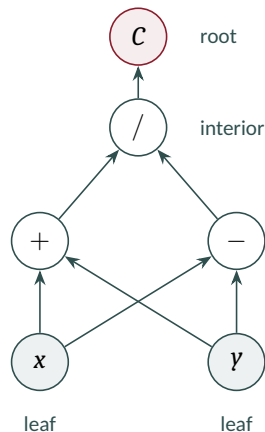
Leaf. An input node, such as x or y . It has no arguments.

Interior node. One primitive operation, such as $+$, $-$ or $/$, applied to its arguments.

Root. The output node. Here it is C .

Edge $u \rightarrow v$. The value u is an argument of v . The edge has the *local derivative* $\partial v / \partial u$.

Evaluation trace. The list of intermediate variables $v_1, v_2, \dots$ in the order they are computed.



$$C = \frac{x + y}{x - y}$$

Terminology (2 of 2): fan-out, fan-in, adjoint

Fan-out of a node. The number of nodes that use it as an argument. Here x and y each have fan-out 2.

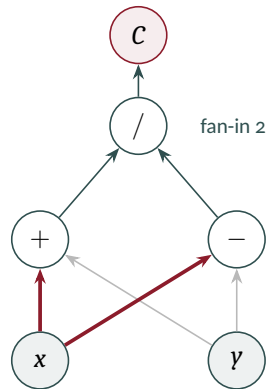
Fan-in of a node. The number of its arguments. A binary operator has fan-in 2.

Path. A chain of edges from a leaf to the root. A node with fan-out 2 opens two paths.

Adjoint of a node v . The quantity $\bar{v} = \partial C / \partial v$. Reverse mode computes adjoints, starting with $\bar{C} = 1$ at the root.

Forward pass. Compute values from the leaves up.

Reverse pass. Compute adjoints from the root down.



Forward pass at $x = 3, y = 1$

Evaluation trace:

$$v_1 = x = 3$$

$$v_2 = y = 1$$

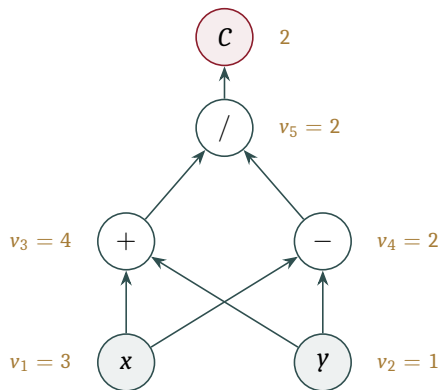
$$v_3 = v_1 + v_2 = 4$$

$$v_4 = v_1 - v_2 = 2$$

$$v_5 = v_3 / v_4 = 2$$

$$C = v_5 = 2$$

Reverse-mode AD stores v_3 and v_4 during the forward pass. The reverse pass will need them.



Local derivatives on the edges

Each operator knows only its own derivative:

$$\frac{\partial v_3}{\partial v_1} = 1,$$

$$\frac{\partial v_3}{\partial v_2} = 1,$$

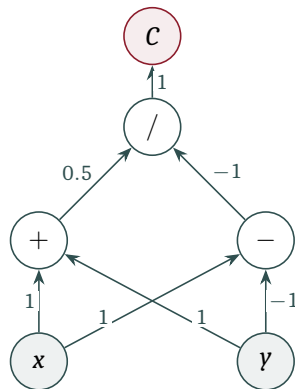
$$\frac{\partial v_4}{\partial v_1} = 1,$$

$$\frac{\partial v_4}{\partial v_2} = -1,$$

$$\frac{\partial v_5}{\partial v_3} = \frac{1}{v_4} = 0.5,$$

$$\frac{\partial v_5}{\partial v_4} = -\frac{v_3}{v_4^2} = -1.$$

The last two use the stored forward values. A node never needs to see the whole graph.



Chain rule: product along a path, sum over paths

x has fan-out 2, so it reaches C by two paths:

$x \rightarrow v_3 \rightarrow v_5$ and $x \rightarrow v_4 \rightarrow v_5$.

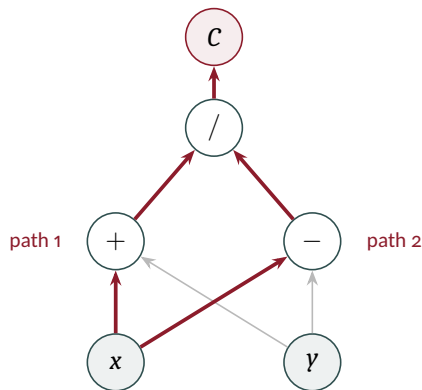
$$\frac{\partial C}{\partial x} = \frac{\partial C}{\partial v_5} \left(\frac{\partial v_5}{\partial v_3} \frac{\partial v_3}{\partial v_1} + \frac{\partial v_5}{\partial v_4} \frac{\partial v_4}{\partial v_1} \right)$$

$$= 1 (0.5 \cdot 1 + (-1) \cdot 1) = -0.5$$

Check: $\frac{\partial C}{\partial x} = \frac{-2y}{(x-y)^2} = -0.5$.

Dropping the second path gives 0.5, with the wrong sign.

A single-path graph doesn't have a sum. The sum appears where a node has fan-out greater than 1.



Reverse pass: adjoints add up at fan-out

Visit nodes from the root down:

1 $\bar{v}_5 = \bar{c} = 1$

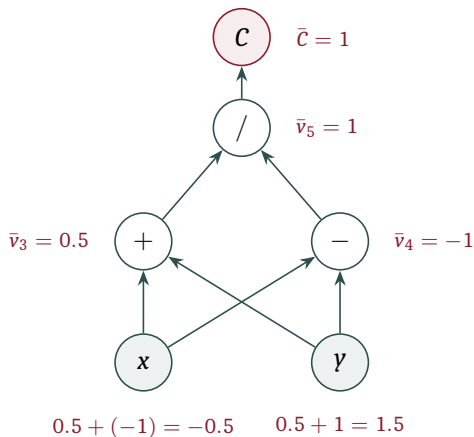
2 $\bar{v}_3 = \bar{v}_5 \cdot \frac{1}{v_4} = 0.5,$
 $\bar{v}_4 = \bar{v}_5 \cdot \left(-\frac{v_3}{v_4^2}\right) = -1$

3 $\bar{x} = \bar{v}_3 \cdot 1 + \bar{v}_4 \cdot 1 = -0.5$

4 $\bar{y} = \bar{v}_3 \cdot 1 + \bar{v}_4 \cdot (-1) = 1.5$

Steps 3 and 4 are the sums. A leaf's adjoint is final only after both of its consumers have reported. So the pass must follow a reverse topological order.

The sum happens once, at the node. Paths are not listed.



The same computation in PyTorch

```
import torch

x = torch.tensor(3.0, requires_grad=True)
y = torch.tensor(1.0, requires_grad=True)

a = x + y          # v3, x and y fan out here
b = x - y          # v4, and here
C = a / b          # v5

a.retain_grad()    # keep adjoints of
b.retain_grad()    # interior nodes

C.backward()        # reverse pass, seed 1

print(C.item())     # 2.0
print(a.grad.item()) # 0.5
print(b.grad.item()) # -1.0
print(x.grad.item()) # -0.5
print(y.grad.item()) # 1.5
```

The printed values match the hand computation line for line.

$x.grad$ holds $\bar{x}$, the sum of the two path contributions.

PyTorch frees the adjoints of interior nodes by default. Use `retain_grad()` to keep them for verification purposes. $\bar{v}_3$ and $\bar{v}_4$.

`backward()` on a scalar seeds the root with $\bar{C} = 1$.

Inside PyTorch autograd: where the sum happens

```
f = C.grad_fn
print(f.name())
# DivBackward0

print([g.name() for g, _ in f.next_functions])
# ['AddBackward0', 'SubBackward0']

gx_add = a.grad_fn.next_functions[0][0]
gx_sub = b.grad_fn.next_functions[0][0]
print(gx_add.name())           # AccumulateGrad
print(gx_add is gx_sub)        # True
```

`grad_fn` is the recorded graph, read from the root down.

Both `AddBackward0` and `SubBackward0` point to the *same* `AccumulateGrad` node for `x`. That shared node is the fan-out, and it adds the two incoming gradients into `x.grad`.

Interior nodes with fan-out get an input buffer that adds incoming gradients before the node's own backward runs.

A different sum: accumulation across calls

```
x = torch.tensor(3.0, requires_grad=True)
y = torch.tensor(1.0, requires_grad=True)

C = (x + y) / (x - y)
C.backward()
print(x.grad.item())    # -0.5

C = (x + y) / (x - y)
C.backward()
print(x.grad.item())    # -1.0, accumulated

x.grad.zero_()          # what zero_grad() does
```

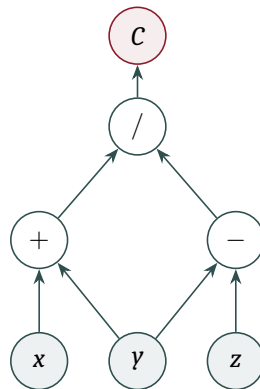
A second `backward()` adds into `x.grad`. It does not overwrite it.

This sum has nothing to do with the chain rule. It is a design choice that lets you accumulate gradients over several mini-batches.

Forgetting `optimizer.zero_grad()` gives gradients that grow with each step and look **plausible**. The chain-rule sum happens inside one pass. This one happens between passes.

Exercise

- 1 Compute $\partial C / \partial y$ by listing both paths. Confirm it equals $2x / (x - y)^2 = 1.5$.
- 2 Change the graph to $C = (x + y) / (z - y)$. What is the fan-out of x now? Does $\partial C / \partial x$ still need a sum?
- 3 What happens in the forward and reverse passes at $x = y$?
- 4 Predict $x.\text{grad}$ after three calls to $C.\text{backward}()$ with no zeroing.



graph for question 2

Autograd System

Autograd System in PyTorch

PyTorch provides automatic differentiation through its autograd system. It means any operations performed on tensors with `requires_grad=True` will automatically compute gradients and build a computational graph dynamically. PyTorch tracks gradients automatically during the forward pass and computes them when `.backward()` is called.

Example 9

```
import torch
# Initialize x with gradient tracking enabled
x = torch.tensor(3.0, requires_grad=True)
# Compute  $y = x^3$ 
y = x**3
# Compute gradient  $dy/dx$ 
y.backward()
# Access the gradient
dy_dx = x.grad
# Convert to numpy if needed
dy_dx.item() # or dy_dx.numpy() for array format
```

What's the output?

Some Other Optimizers

Gradient Descent is one of the many optimizers that can be used to optimize a function and find its minima. Some other commonly popular optimizers are

- 1 RMSProp
- 2 Adagrad (Adaptive Gradient)
- 3 SGD with Momentum
- 4 SGD with Nesterov Accelerated Gradient
- 5 AdaDelta
- 6 ADAM

RMSProp

RMSProp (Root Mean Square Propagation) is an adaptive learning rate method that divides the learning rate by an exponentially decaying average of squared gradients.

⚡ Update rule:

$$\begin{aligned}g_t &= \nabla_{\theta} J(\theta_t) \\v_t &= \beta v_{t-1} + (1 - \beta) g_t^2 \\ \theta_{t+1} &= \theta_t - \frac{\eta}{\sqrt{v_t} + \epsilon} g_t\end{aligned}\tag{23}$$

⚡ η : Learning rate

⚡ β : Decay rate (typically 0.9)

⚡ ϵ : Small constant for numerical stability

Adagrad (Adaptive Gradient)

Adagrad adapts the learning rate to the parameters, performing smaller updates for frequently occurring features and larger updates for infrequent ones.

⚡ Update rule:

$$\begin{aligned}g_t &= \nabla_{\theta} J(\theta_t) \\G_t &= G_{t-1} + g_t^2 \\ \theta_{t+1} &= \theta_t - \frac{\eta}{\sqrt{G_t} + \epsilon} g_t\end{aligned}\tag{24}$$

⚡ η : Learning rate

⚡ G_t : Accumulated squared gradients

⚡ ϵ : Small constant for numerical stability

SGD with Momentum

SGD with Momentum accelerates gradient descent by adding a fraction of the previous update to the current update.

⚡ Update rule:

$$\begin{aligned}g_t &= \nabla_{\theta} J(\theta_t) \\v_t &= \gamma v_{t-1} + \eta g_t \\ \theta_{t+1} &= \theta_t - v_t\end{aligned}\tag{25}$$

⚡ η : Learning rate

⚡ γ : Momentum term (typically 0.9)

⚡ v_t : Velocity vector

SGD with Nesterov Accelerated Gradient

Nesterov Accelerated Gradient (NAG) improves momentum by calculating the gradient at the estimated future position.

⚡ Update rule:

$$\begin{aligned}g_t &= \nabla_{\theta} J(\theta_t - \gamma \mathbf{v}_{t-1}) \\ \mathbf{v}_t &= \gamma \mathbf{v}_{t-1} + \eta g_t \\ \theta_{t+1} &= \theta_t - \mathbf{v}_t\end{aligned}\tag{26}$$

⚡ η : Learning rate

⚡ γ : Momentum term (typically 0.9)

⚡ $\mathbf{v}_t$: Velocity vector

AdaDelta

AdaDelta is an extension of Adagrad that reduces its aggressive, monotonically decreasing learning rate.

⚡ Update rule:

$$\begin{aligned}g_t &= \nabla_{\theta} J(\theta_t) \\E[g^2]_t &= \rho E[g^2]_{t-1} + (1 - \rho) g_t^2 \\ \Delta \theta_t &= - \frac{\sqrt{E[\Delta \theta^2]_{t-1} + \epsilon}}{\sqrt{E[g^2]_t + \epsilon}} g_t \\E[\Delta \theta^2]_t &= \rho E[\Delta \theta^2]_{t-1} + (1 - \rho) \Delta \theta_t^2 \\ \theta_{t+1} &= \theta_t + \Delta \theta_t\end{aligned}$$

(27)

⚡ ρ : Decay rate (typically 0.9)

⚡ ϵ : Small constant for numerical stability

ADAM (Adaptive Moment Estimation)

ADAM combines the benefits of RMSProp and Momentum by adapting learning rates and using momentum.

⚡ Update rule:

$$\begin{aligned}g_t &= \nabla_{\theta} J(\theta_t) \\m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t \\v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \\\hat{m}_t &= \frac{m_t}{1 - \beta_1^t} \\\hat{v}_t &= \frac{v_t}{1 - \beta_2^t} \\\theta_{t+1} &= \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t\end{aligned}\tag{28}$$

⚡ η : Learning rate

⚡ β_1, β_2 : Exponential decay rates (typically 0.9 and 0.999)

⚡ ϵ : Small constant for numerical stability

Constraint Optimization with Inequality

Inequality Constraints

$$\begin{array}{l} \max / \min f(\mathbf{x}) \\ \text{subject to} \\ g(\mathbf{x}) \geq 0 \end{array}$$

Without the constraint, $\frac{df(\mathbf{x}^*)}{d\mathbf{x}} = 0$.

When the inequality constraint is added in, either the solution could occur when $g(\mathbf{x}^*) = 0$, or $g(\mathbf{x}^*) > 0$.

Inequality Constraints with Lagrange Multiplier

We could still write the Lagrangian:

$$\begin{aligned}\mathcal{L}(\mathbf{x}, \lambda) &= f(\mathbf{x}) + \lambda g(\mathbf{x}) \\ \frac{\partial \mathcal{L}}{\partial \mathbf{x}}(\mathbf{x}^*, \lambda^*) &= \frac{\partial f}{\partial \mathbf{x}}(\mathbf{x}^*) + \frac{\partial g(\mathbf{x}^*)'}{\partial \mathbf{x}} \lambda^* = 0 \\ \frac{\partial \mathcal{L}}{\partial \lambda}(\mathbf{x}^*, \lambda^*) &= g(\mathbf{x}^*) \geq 0 \\ \Rightarrow \lambda^* \geq 0, \quad \lambda^* \odot \frac{\partial \mathcal{L}}{\partial \lambda}(\mathbf{x}^*, \lambda^*) &= \lambda^* \odot g(\mathbf{x}^*) = 0\end{aligned}\tag{29}$$

where $\odot$ is the element-wise product.

These conditions are called **Karush-Kuhn-Tucker Conditions (KKT)**.

Karush-Kuhn-Tucker Conditions

⚡ If we have several equality and inequality constraints, in the following form

$$\arg \min_{\mathbf{x}} f(\mathbf{x}), \text{ s.t. } g_i(\mathbf{x}) = 0, h_j(\mathbf{x}) \leq 0 \quad (30)$$

The necessary and sufficient conditions (also known as the KKT conditions) for

$$\begin{aligned} \frac{\partial L(\mathbf{x}^*, \lambda^*, \beta^*)}{\partial \mathbf{x}} = 0, \quad \frac{\partial L(\mathbf{x}^*, \lambda^*, \beta^*)}{\partial \beta} = 0, \quad g_i(\mathbf{x}^*) = 0, \quad h_j(\mathbf{x}^*) \leq 0, \\ \sum_{j=1}^J \beta_j^* h_j(\mathbf{x}^*) = 0, \quad \lambda_i, \beta_j \geq 0 \end{aligned} \quad (31)$$

The last condition is sometimes written in the equivalent form: $\beta_j^* h_j(\mathbf{x}^*) = 0$.

The Generalized Lagrangian

The generalized Lagrangian function to **minimize** $f(x)$ subject to the equality constraints on $g_i(x)$ and inequality constraints on $h_j(x)$ is given by:

$$L(x, \lambda, \beta) = f(x) + \sum_{i=1}^I \lambda_i g_i(x) + \sum_{j=1}^J \beta_j h_j(x) \quad (32)$$

Example 1

$$\min f(\mathbf{x}) = x_1^2 + x_2^2 \quad (33)$$

subject to

$$\begin{aligned} x_1 + x_2 &= 1 \\ x_2 &\leq \alpha \end{aligned} \quad (34)$$

Example 1

Class Work

Example 1

Class Work

Example 1

Class Work

Example 2

$$\min f(\mathbf{x}) = (x_1 - 2)^2 + (x_2 - 1)^2 \quad (35)$$

subject to

$$\begin{aligned} h_1(\mathbf{x}) &= x_1^2 - x_2 \leq 0 \\ h_2(\mathbf{x}) &= x_1 + x_2 - 2 \leq 0 \end{aligned} \quad (36)$$

Example 2

Class Work

Example 2

Class Work

Example 2

Class Work

Example 2

Class Work

Example 2

Class Work

Further Reading

- ⚡ <https://neos-guide.org/guide/algorithms/gradient-projection/>
- ⚡ <https://www.stat.cmu.edu/~siva/teaching/725/lec6.pdf>
- ⚡ <https://www.cs.ubc.ca/~schmidtm/Courses/5XX-S20/S5.pdf>

Python Notebook

https://github.com/rahulbhadani/CPE486586_FA26/blob/master/Code/CPE486586_Ch05_Optimization_and_Gradient_Descent.ipynb

The End